

Montor

Aplicación web de finanzas personales multi-moneda con asistencia de IA

Nicolás Ploskinos

Septiembre de 2026

Índice

Resumen	2
1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	3
3. Descripción funcional del sistema	3
3.1. Gestión de transacciones	3
3.2. Multi-moneda, cotización oficial y valor histórico	3
3.3. Viajes	4
3.4. Presupuestos y gastos recurrentes	4
3.5. Importación de movimientos bancarios	4
3.6. Perfil y autogestión de la suscripción	5
4. Arquitectura del sistema	5
4.1. Stack tecnológico	5
4.2. Identidad visual y accesibilidad de la interfaz	6
4.3. Organización del código	6
4.4. Modelo de datos	6
5. Integración de Inteligencia Artificial	7
5.1. Resumen mensual generado por IA	7
5.2. Bot conversacional de WhatsApp	7
5.3. Consideraciones de costo y confiabilidad	8
6. Seguridad	8
7. Testing y aseguramiento de calidad	9
8. Integración continua	9
9. Despliegue e infraestructura	9
10. Modelo de negocio	10
11. Conclusiones y trabajo futuro	10
Referencias / tecnologías utilizadas	11

Autor: Nicolás Ploskinos

Fecha: Septiembre de 2026

Resumen

Este trabajo presenta Montor, una aplicación web de control de gastos e ingresos personales pensada específicamente para el contexto económico argentino, donde conviven distintas monedas (pesos, dólares y euros) en el día a día de una misma persona. El sistema permite registrar transacciones, organizarlas por viajes, definir presupuestos por categoría, automatizar gastos recurrentes, e interactuar con la aplicación tanto desde una interfaz web como desde WhatsApp mediante un asistente conversacional impulsado por modelos de lenguaje (LLM). Se describen las decisiones de arquitectura —incluida la migración del frontend desde plantillas servidas por el servidor hacia una aplicación de página única en React—, la integración de Inteligencia Artificial en distintos puntos del producto, las medidas de seguridad implementadas, la estrategia de testing automatizado y de integración continua, y el modelo de despliegue e infraestructura utilizado. El proyecto fue desarrollado y mantenido por un único desarrollador, e incorpora un modelo de negocio freemium con un plan pago (Pro) gestionado a través de Mercado Pago.

Palabras clave: finanzas personales, aplicación web, Flask, React, Inteligencia Artificial, procesamiento de lenguaje natural, WhatsApp Business API, Supabase, testing automatizado.

1. Introducción

El manejo de las finanzas personales en Argentina presenta una particularidad poco frecuente en otros países: es habitual que una misma persona perciba ingresos en pesos y en dólares (por ejemplo, a través de trabajo freelance o en relación de dependencia con parte del sueldo dolarizado), y que sus gastos también se distribuyan entre ambas monedas. Las aplicaciones de finanzas personales genéricas disponibles en el mercado —tanto locales como internacionales— rara vez contemplan este escenario de forma nativa: suelen asumir una única moneda de referencia, o tratan la conversión de divisas como una funcionalidad secundaria.

Montor nace para resolver este problema concreto: permitir que una persona lleve el control de su economía personal viendo, en todo momento, cuánto tiene y cuánto gasta, independientemente de en qué moneda haya ocurrido cada movimiento, y con la cotización oficial actualizada automáticamente.

A partir de esa base, el proyecto fue creciendo para incorporar funcionalidades adicionales que resuelven fricciones reales del uso cotidiano: el registro de gastos durante un viaje al exterior (con seguimiento consolidado en dólares), el establecimiento de presupuestos por categoría, la automatización de gastos que se repiten mes a mes, y —el aporte más significativo del proyecto— la posibilidad de cargar y consultar movimientos conversando directamente por WhatsApp, en lenguaje natural, en español o en inglés, con o sin estructura fija, incluso por nota de voz.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación web de gestión de finanzas personales que resuelva de forma nativa el manejo simultáneo de múltiples monedas, minimizando la fricción de carga de datos mediante interfaces conversacionales asistidas por Inteligencia Artificial.

2.2. Objetivos específicos

- Permitir el registro, edición y eliminación de transacciones (gastos e ingresos) en pesos argentinos, dólares estadounidenses y euros, con conversión automática entre monedas usando la cotización oficial del Banco Nación.
- Ofrecer una vista consolidada del balance financiero del usuario, independientemente de la moneda en que se registró cada movimiento.
- Permitir agrupar gastos bajo un "viaje", con seguimiento del gasto total convertido a dólares.
- Permitir definir presupuestos mensuales por categoría, con alertas visuales de proximidad o exceso.
- Automatizar la carga de gastos e ingresos recurrentes (alquiler, suscripciones, sueldo).
- Incorporar un canal de interacción alternativo a la interfaz web, vía WhatsApp, capaz de interpretar lenguaje natural (incluyendo jerga rioplatense, mensajes de voz, y both español e inglés) para registrar movimientos y responder preguntas sobre las finanzas del usuario.
- Generar, mediante IA, un resumen en lenguaje natural del desempeño financiero mensual del usuario.
- Garantizar niveles razonables de seguridad, privacidad y calidad de software dentro del alcance de un proyecto desarrollado por una única persona.
- Sustener el desarrollo del producto mediante un modelo de suscripción (freemium) que permita cubrir los costos de infraestructura y APIs de terceros.

3. Descripción funcional del sistema

3.1. Gestión de transacciones

El núcleo de la aplicación es el registro de transacciones (gastos e ingresos), cada una con monto, moneda (ARS/USD/EUR), categoría, descripción y fecha. El usuario puede cargar, editar y eliminar movimientos desde una interfaz web responsive, buscar y filtrar por texto libre, categoría o rango de fechas, y exportar su historial completo a Excel, PDF o CSV. En móvil, además, cada movimiento se puede eliminar deslizándolo hacia el costado.

La interfaz se organiza distinto según el dispositivo. En escritorio ocupa el ancho completo de la pantalla y se divide en tres columnas: navegación a la izquierda, la lista de movimientos con sus filtros al centro, y una columna de widgets a la derecha (presupuestos del mes, resumen mensual, gastos recurrentes, vinculación de WhatsApp y cotizaciones), todos plegables y cerrados por defecto para que el usuario abra únicamente lo que quiere mirar. En móvil, ese mismo contenido se apila en una sola columna encabezada por el balance consolidado, la lista se limita a los movimientos más recientes con un botón para ver el resto, y la navegación pasa a una barra inferior fija con las mismas secciones que la barra lateral de escritorio.

3.2. Multi-moneda, cotización oficial y valor histórico

Todas las conversiones entre monedas se resuelven mediante la cotización oficial del Banco Nación, obtenida de una API pública (bluelytics.com.ar) y cacheada en el servidor durante ventanas cortas de tiempo para minimizar la dependencia de un servicio externo en cada request. Si la cotización no puede obtenerse en un momento dado, el sistema degrada de forma controlada (mostrando la información disponible sin romper el resto de la aplicación) en lugar de fallar por completo.

Un problema específico del dominio, detectado durante el uso real de la aplicación, es que un gasto en dólares o euros no debería revalorizarse cada vez que cambia la cotización: si un usuario gastó USD 300 el 11 de agosto con el dólar a \$1.500, ese gasto tiene que seguir valiendo \$450.000 el 30 de agosto aunque el dólar haya subido a \$1.550 para esa fecha. Para resolver esto, cada transacción guarda una "foto" de la cotización del día en que se carga —no solo para su propia moneda, sino también para las otras dos, ya que un viaje puede necesitar convertir un gasto en pesos a dólares más adelante— y todos los cálculos de totales (resumen mensual, viajes, balance consolidado) usan esa foto en lugar de la cotización del momento en que se consulta.

Esto plantea un caso adicional: la carga tardía, cuando el usuario registra hoy un movimiento que ocurrió días atrás. Para esos casos, el sistema consulta la serie histórica de cotización oficial del dólar que publica la misma API (disponible desde 2011), buscando la fecha exacta del movimiento; si esa fecha cae en un fin de semana o feriado —días sin publicación—, retrocede día por día hasta encontrar la cotización publicada más cercana. La cotización histórica del euro no está disponible públicamente en ninguna API gratuita conocida, por lo que se aproxima aplicando al dólar histórico encontrado la relación EUR/USD de la cotización del día en que se hace la carga —una aproximación explícita y documentada, no un valor exacto de esa fecha pasada—. Las transacciones cargadas antes de que existiera este mecanismo no tienen esta foto guardada, y para ellas el sistema usa la cotización actual como mejor esfuerzo, sin romper con datos previos.

3.3. Viajes

Los usuarios pueden crear "viajes" (con nombre y rango de fechas) y etiquetar transacciones existentes o nuevas como pertenecientes a ese viaje, sin importar en qué moneda se hayan cargado originalmente. El sistema calcula el gasto total del viaje convertido simultáneamente a pesos, dólares y euros, y ofrece un desglose por categoría con gráfico.

3.4. Presupuestos y gastos recurrentes

El usuario puede definir un presupuesto mensual por categoría (y moneda), visualizado como una barra de progreso que cambia de color según el porcentaje consumido, con un ícono de alerta como respaldo no dependiente del color (relevante para usuarios con daltonismo). Los gastos recurrentes (por ejemplo, alquiler o suscripciones) se configuran una única vez, indicando su frecuencia (semanal o mensual), y el sistema los genera automáticamente en cada período sin intervención manual.

3.5. Importación de movimientos bancarios

La aplicación permite subir un archivo CSV exportado desde un banco o billetera virtual. El sistema detecta automáticamente el delimitador, reconoce las columnas relevantes (fecha, monto,

descripción, o columnas separadas de débito/crédito) mediante heurísticas de coincidencia de encabezados, y le muestra al usuario una vista previa editable antes de confirmar la importación masiva.

3.6. Perfil y autogestión de la suscripción

Desde su perfil, el usuario administra su propia cuenta sin intervención del desarrollador: puede cambiar su nombre de usuario (validando que no esté tomado por otra cuenta), cambiar su contraseña (verificando primero la actual), y ver el estado de su plan. Si tiene una suscripción Pro activa, ve además el ciclo contratado (mensual o anual) y la fecha del próximo cobro, y puede cambiar el medio de pago o dar de baja la suscripción.

El cambio de medio de pago merece una aclaración de diseño: como el procesador de pagos no expone una forma de reemplazar la tarjeta de una suscripción ya autorizada, la operación se resuelve creando una preaprobación nueva —donde el usuario carga la tarjeta directamente en el entorno del procesador, sin que la aplicación vea nunca los datos de la tarjeta— y cancelando la anterior. Para poder ofrecer todo esto, la aplicación guarda el identificador de la preaprobación de Mercado Pago asociada a cada usuario, el ciclo contratado y la fecha del próximo cobro informada por el procesador.

4. Arquitectura del sistema

4.1. Stack tecnológico

El backend está desarrollado en Python con el framework Flask, y el frontend es una aplicación de página única (SPA) construida con React, TypeScript y Vite.

El proyecto no nació así: la primera versión servía las vistas como plantillas Jinja2 con HTML, CSS y JavaScript "vanilla" embebidos, sin dependencias de Node.js ni proceso de compilación —una decisión deliberada que permitía iterar sobre la interfaz sin fricción de tooling adicional—. A medida que la interfaz ganó estado propio (filtros combinados, acordeones, modales, gráficos que reaccionan al tema claro/oscuro, animaciones), el costo de sostener ese estado a mano en JavaScript sin componentes superó al costo del tooling, y se realizó una migración completa del frontend a React, página por página, manteniendo la aplicación funcionando en cada paso.

Tras esa migración, la división de responsabilidades quedó así:

- Flask expone exclusivamente una API JSON y sirve un shell HTML genérico (una única plantilla) para todas las rutas de la aplicación; ya no genera marcado de interfaz.
- React Router resuelve la navegación del lado del cliente.
- TanStack Query gestiona el ciclo de vida de los datos del servidor (cacheo, revalidación e invalidación tras cada escritura), eliminando el estado de sincronización manual que antes había que escribir a mano en cada pantalla.
- Los estilos usan CSS Modules, que acotan cada regla al componente que la declara y evitan las colisiones de nombres propias de una hoja de estilos global.

Una particularidad del despliegue condiciona este esquema: el hosting utilizado no ejecuta Node.js, por lo que la compilación se hace localmente y el resultado compilado se versiona en el repositorio

junto al código fuente, quedando servido por Flask como archivos estáticos. Es un compromiso explícito —versionar un artefacto derivado no es una práctica deseable en general— aceptado para poder usar un stack de frontend moderno sobre un hosting gratuito sin pipeline de build propio.

4.2. Identidad visual y accesibilidad de la interfaz

La interfaz utiliza un sistema de diseño propio con soporte de modo claro y oscuro, donde ambos modos se definen como dos juegos de variables CSS sobre los mismos nombres de token (superficies, texto, bordes y colores semánticos de ingreso/gasto/alerta). Los componentes nunca referencian un color literal: leen el token, de modo que agregar o ajustar un modo no requiere tocar cada componente.

La disposición se adapta al dispositivo: en escritorio, una barra lateral de navegación permanente junto a una grilla de tres columnas (navegación, contenido principal y panel de widgets); en móvil, el mismo conjunto de secciones se presenta como una barra de navegación inferior fija con ícono y etiqueta, respetando el área segura del dispositivo (el espacio reservado al indicador de gestos en teléfonos sin botón físico). Los gráficos leen los colores resueltos del tema en tiempo de ejecución, de forma que acompañan el cambio entre modo claro y oscuro sin duplicar configuración.

La persistencia de datos se resuelve con Supabase (una capa de Postgres gestionado con una API REST autogenerada), accedida mediante su cliente Python oficial. La autenticación de usuarios es propia (no se utiliza Supabase Auth): las contraseñas se almacenan hasheadas con `werkzeug.security`, y la sesión del usuario se maneja mediante las cookies de sesión firmadas de Flask.

4.3. Organización del código

El backend evolucionó de un único archivo monolítico de aproximadamente 1300 líneas a una estructura modular organizada por dominio funcional, utilizando Blueprints de Flask:

- `monitor_server.py`: módulo "núcleo" — instancia de la aplicación Flask, clientes de Supabase y de la API de Gemini, funciones auxiliares compartidas (autenticación, conversión de moneda, lógica del bot de WhatsApp, cálculo de estadísticas) y el registro final de los distintos blueprints.
- `routes_paginas.py`: páginas HTML (landing, dashboard, login).
- `routes_auth.py`: registro e inicio de sesión.
- `routes_monitor.py`: transacciones, exportación/importación, presupuestos, recurrentes, estadísticas y resumen con IA.
- `routes_viajes.py`: gestión de viajes.
- `routes_whatsapp.py`: vinculación y webhook del bot de WhatsApp.
- `routes_pagos.py`: suscripción Pro vía Mercado Pago.
- `routes_cuenta.py`: perfil del usuario — cambio de nombre y contraseña, estado de la suscripción y baja.

Cada módulo de rutas accede a los recursos compartidos importando el módulo núcleo completo (en lugar de importar símbolos individuales), un patrón elegido deliberadamente para que la suite de tests pudiera seguir reemplazando dependencias (cliente de base de datos, cliente de IA) por mocks sin importar en qué archivo terminara viviendo el código que las utiliza.

4.4. Modelo de datos

Las principales entidades del modelo de datos son: usuarios, transacciones, viajes, presupuestos, recurrentes y whatsapp_users (esta última vincula un número de teléfono con una cuenta de usuario, y almacena el identificador de la última transacción cargada por ese medio, para soportar la función de deshacer la última carga).

La tabla usuarios guarda, además de las credenciales y el plan vigente, los datos necesarios para la autogestión de la suscripción descrita en la sección 3.6: el identificador de la preaprobación de Mercado Pago, el ciclo contratado y la fecha del próximo cobro. Estos dos últimos se actualizan desde el webhook del procesador de pagos, de modo que la aplicación no necesita consultar un servicio externo cada vez que el usuario abre su perfil.

5. Integración de Inteligencia Artificial

La incorporación de modelos de lenguaje (LLM) es uno de los ejes centrales del proyecto, aplicada en tres puntos distintos del producto, todos mediante la API de Google Gemini:

5.1. Resumen mensual generado por IA

A partir de los datos ya calculados de gastos, ingresos y categorías del mes (y su comparación con el mes anterior), el sistema construye un prompt con esa información y solicita al modelo un párrafo breve, en lenguaje natural, que interprete la situación financiera del usuario en lugar de limitarse a repetir las cifras. El resumen se genera en el mismo idioma en que el usuario esté navegando la aplicación (español o inglés), y se cachea por usuario, mes y año para evitar regenerar contenido idéntico en cada visita — excepto para el mes en curso, que se recalcula siempre por tratarse de datos que aún pueden cambiar.

5.2. Bot conversacional de WhatsApp

Este es el componente de mayor complejidad técnica del proyecto. A través de la API de WhatsApp Business (Meta Cloud API), el sistema recibe mensajes de texto, notas de voz, y comandos especiales, y los procesa de la siguiente manera:

- Interpretación de intención por IA: cada mensaje entrante se envía a Gemini junto con un esquema de salida estructurada (JSON Schema) que le exige al modelo clasificar el mensaje en una de tres intenciones —carga de un movimiento, pedido de borrar la última carga, u otra cosa (pregunta o charla)— y, en el primer caso, extraer tipo, monto, moneda, categoría y descripción. El diseño evita depender de frases exactas: expresiones como "epa, me equivoqué, bórralo" o "undo that" se reconocen correctamente sin necesidad de que coincidan con un patrón predefinido.
- Reconocimiento de voz: los audios se descargan desde los servidores de Meta (que solo entregan un identificador de medio, requiriendo una segunda solicitud para obtener la URL temporal del archivo) y se transcriben directamente con Gemini, que admite entrada de audio de forma nativa, sin necesidad de un servicio de reconocimiento de voz independiente.
- Soporte bilingüe: el modelo detecta el idioma del mensaje (español o inglés) como parte de la misma clasificación, y todas las respuestas del bot —confirmaciones, errores, el resumen conversacional— se generan en ese mismo idioma.

- Chat de preguntas sobre los propios datos: si el mensaje no resulta ser ni una carga ni un pedido de borrado, se le envían al modelo los movimientos de los últimos seis meses del usuario como contexto, junto con la fecha actual (incluyendo el día de la semana, para poder resolver referencias relativas como "el viernes pasado"), y se le pide una respuesta breve basada exclusivamente en esos datos.
- Degradación controlada: si la API de IA no está disponible (sin conexión, error, o tiempo de espera agotado), el sistema no deja de funcionar: cae a un conjunto de reglas fijas basadas en expresiones regulares que reconocen un subconjunto más limitado, pero funcional, de mensajes en español e inglés.

5.3. Consideraciones de costo y confiabilidad

Durante el desarrollo se evaluaron distintos modelos de la familia Gemini en función de su cuota gratuita disponible, priorizando modelos de la línea "Flash-Lite" —de menor costo computacional— por sobre modelos de mayor capacidad, dado que las tareas involucradas (clasificación de intención, extracción de campos estructurados, resúmenes breves) no requieren el nivel de razonamiento de un modelo de mayor porte. Esta decisión resultó, en la práctica, un compromiso razonable entre calidad de respuesta y límites de uso gratuito.

6. Seguridad

Se implementaron las siguientes medidas, revisadas específicamente en el marco de este trabajo:

- Contraseñas: almacenadas exclusivamente como hash (nunca en texto plano), utilizando las funciones de `werkzeug.security`, con un mínimo de 8 caracteres exigido en el registro.
- Protección contra fuerza bruta: el endpoint de inicio de sesión bloquea temporalmente (15 minutos) los intentos desde un mismo nombre de usuario luego de 5 intentos fallidos consecutivos.
- Cookies de sesión: configuradas explícitamente con los atributos `Secure` (solo se transmiten por HTTPS), `HttpOnly` (inaccesibles desde JavaScript) y `SameSite=Lax` (mitigación de ataques CSRF).
- Row Level Security (RLS): habilitado en todas las tablas de la base de datos en Supabase. Dado que el sistema no utiliza el mecanismo de autenticación nativo de Supabase (`auth.uid()`), la separación de datos por usuario se aplica en la capa de aplicación (a través de la sesión de Flask); RLS se utiliza aquí como una capa de defensa adicional que impide el acceso directo a las tablas si la clave de API llegara a exponerse, más que como el mecanismo primario de aislamiento por usuario — una limitación arquitectónica reconocida y documentada.
- Aislamiento de datos entre cuentas: se auditó y eliminó un remanente de la versión de usuario único original de la aplicación (previa al modelo multi-usuario) que, bajo ciertas condiciones, podía asociar transacciones sin dueño a la cuenta que se registrara a continuación — una revisión concreta de que los datos de un usuario nunca queden expuestos a otro.
- Límite de tamaño de subida: se configuró un límite global de 5 MB para el cuerpo de las solicitudes HTTP, además del límite específico ya existente para los archivos CSV de importación.
- Verificación de firma de webhooks: las solicitudes entrantes del webhook de WhatsApp se

validan mediante HMAC-SHA256 contra el secreto de aplicación provisto por Meta, evitando que solicitudes falsificadas puedan inyectar mensajes.

- Restricción del bot de WhatsApp a una única cuenta: dado que la aplicación de Meta permanece en modo de desarrollo (pendiente del proceso de verificación de negocio de Meta), la API solo puede entregar mensajes a números de teléfono explícitamente autorizados. El sistema refuerza esta restricción también del lado propio, ocultando la funcionalidad en la interfaz y rechazando en el backend cualquier intento de vinculación que no corresponda a la cuenta autorizada.
- Manejo de errores: se agregaron manejadores globales para errores 404 y 500 que responden de forma controlada (JSON prolijo para la API, una página con estilo propio para el resto) en lugar de exponer la página de error genérica del framework.

Además, dado que la aplicación procesa datos financieros personales y pagos, se publicaron una Política de Privacidad y unos Términos de Servicio (accesibles desde el pie de la landing y enlazados en el registro de cuenta) que detallan qué datos se recolectan, con qué terceros se comparten (Supabase, Google Gemini, Meta, Mercado Pago) y los derechos de acceso, rectificación y supresión que la Ley 25.326 de Protección de Datos Personales reconoce a los usuarios en Argentina.

7. Testing y aseguramiento de calidad

Se desarrolló una suite de casi 150 pruebas automatizadas con pytest, que cubre: autenticación (incluyendo el caso específico de que un intento fallido no revele si el nombre de usuario existe o no, y la exigencia de contraseñas de al menos 8 caracteres), el límite de intentos de inicio de sesión, el acceso denegado a rutas protegidas sin sesión iniciada, las funciones puras de cálculo (conversión de moneda, cómputo de fechas, resolución de la cotización histórica de una fecha pasada), la interpretación de mensajes de WhatsApp (incluyendo casos límite como jerga, distintos idiomas, y la degradación cuando la IA no está disponible), el manejo de errores 404/500, y los distintos endpoints de la API — entre ellos, que el total de un mes o de un viaje no cambie si la cotización del dólar cambia después de cargada una transacción.

Ninguna prueba se conecta a la base de datos real ni a ninguna API externa: tanto el cliente de Supabase como el cliente de Gemini se reemplazan por objetos simulados (fakes) definidos en el archivo de configuración compartido de pytest, lo que permite que la suite completa se ejecute en pocos segundos, de forma determinística y sin credenciales reales.

Se incorporó además ruff como herramienta de análisis estático (linter), configurado con un conjunto de reglas conservador orientado a detectar errores reales (variables o importaciones no utilizadas, nombres indefinidos) sin imponer opiniones de estilo no esenciales.

8. Integración continua

Se configuró un flujo de trabajo de GitHub Actions que se ejecuta automáticamente en cada push o pull request sobre la rama principal, y que corre, en orden, el linter y la suite completa de tests. Esto permite detectar errores de forma temprana, antes de que lleguen a producción, y deja un registro visible (a través de una insignia en el repositorio) del estado de salud del proyecto en todo momento.

9. Despliegue e infraestructura

La aplicación está desplegada en PythonAnywhere, sobre un plan gratuito. El despliegue no es automático: es necesario actualizar el código manualmente (mediante git pull en una consola remota) y recargar la aplicación web luego de cada cambio, una limitación propia del nivel gratuito del servicio de hosting elegido. Las variables de entorno sensibles (claves de API, credenciales de base de datos, tokens de las distintas integraciones) se gestionan mediante un archivo .env que nunca se versiona en el repositorio.

Para el entorno de desarrollo local se corrigió además un problema de arranque propio de Python: al ejecutar el archivo principal directamente, el intérprete lo registra bajo el nombre especial `__main__` en lugar de `montor_server`, lo que hacía que los módulos de rutas —que se importan a sí mismos como `montor_server`— terminaran re-ejecutando todo el archivo desde cero como una segunda instancia independiente, y fallando al registrar los blueprints. Se resolvió registrando explícitamente el módulo bajo ambos nombres al arrancar, un ajuste mínimo que no cambia el comportamiento en producción (donde ya se ejecuta correctamente vía gunicorn).

10. Modelo de negocio

El producto sigue un modelo freemium: el uso básico es gratuito y permanente (hasta 50 transacciones por mes, con historial de los últimos tres meses), mientras que un plan pago (Montor Pro) desbloquea transacciones ilimitadas, historial completo, viajes, gastos recurrentes automáticos, exportación avanzada, importación bancaria y el resumen mensual con IA. La suscripción se gestiona mediante la integración con Mercado Pago, con opción de pago mensual o anual (este último con un descuento equivalente a dos meses gratuitos), y todo usuario nuevo accede a una prueba gratuita de 7 días con las funciones Pro habilitadas. La alta, el cambio de medio de pago y la baja son autogestionables por el propio usuario desde su perfil (ver sección 3.6), sin necesidad de contactar al desarrollador.

11. Conclusiones y trabajo futuro

Montor demuestra que es posible construir, con recursos de un único desarrollador, un producto de software con funcionalidad real y diferenciada —particularmente en su integración de Inteligencia Artificial conversacional multicanal— sin resignar prácticas de ingeniería de software habitualmente asociadas a equipos más grandes: testing automatizado, integración continua, control de versiones, y una arquitectura modular que facilita el mantenimiento a largo plazo.

El proyecto también ilustra que las decisiones de arquitectura tienen fecha de vencimiento: servir la interfaz como plantillas sin proceso de compilación fue la decisión correcta mientras la interfaz era simple, y dejó de serlo cuando el estado del lado del cliente creció; la migración posterior a un framework de componentes se hizo de forma incremental, página por página, con la aplicación funcionando en todo momento.

Entre las líneas de trabajo futuro identificadas se destacan: completar el proceso de verificación de negocio de Meta para habilitar el bot de WhatsApp a cualquier usuario (actualmente restringido a una única cuenta por una limitación de la plataforma, no del sistema propio); incorporar

categorización automática de transacciones mediante IA en la carga desde la interfaz web (hoy disponible únicamente vía WhatsApp); y dividir el paquete compilado del frontend en fragmentos cargados bajo demanda, hoy servido como un único archivo que supera el umbral recomendado por la herramienta de compilación.

Referencias / tecnologías utilizadas

- Flask (API web, Python)
- React + TypeScript (interfaz de usuario)
- Vite (compilación del frontend)
- React Router (navegación del lado del cliente)
- TanStack Query (gestión del estado del servidor en el cliente)
- Supabase (Postgres gestionado, API REST)
- Google Gemini API (modelos gemini-3.5-flash-lite) — interpretación de lenguaje natural, transcripción de audio, generación de texto
- WhatsApp Business Platform (Meta Cloud API)
- Mercado Pago (procesamiento de pagos, suscripciones recurrentes)
- pytest (testing automatizado)
- ruff (análisis estático de código)
- GitHub Actions (integración continua)
- PythonAnywhere (hosting)
- Chart.js (visualización de datos en el frontend)